

Experiment 3: Implementation of Remote Method Invocation (RMI)

AIM:

To learn the concepts of Remote Procedure Call and Remote Method Invocation, develop applications which uses Distributed Objects and learn the working and significance of stub object (Client side) and Skeleton objects (Server side)

PROCEDURE:

1. Create three files: MyInterface.java, MyServer.java and MyClient.java as given.
2. Open a command prompt, go to the folder containing the files and compile all programs using `javac *.java`
3. Start the RMI registry by typing `start rmiregistry` (a new blank window will open – **do not close it**).
4. In the same command prompt start the server by typing `java MyServer` You should see the message **Server is Ready...**
5. Open a second command prompt (or use another system) and run the client as `java MyClient localhost 200 300 100`
6. The client will display the three addition results:
7. Keep the rmiregistry window and the server window open until the demonstration is finished.
8. To stop the experiment, simply close the server and registry windows.

FLOW CHART:

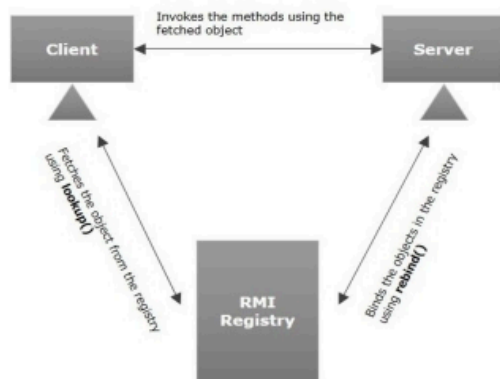
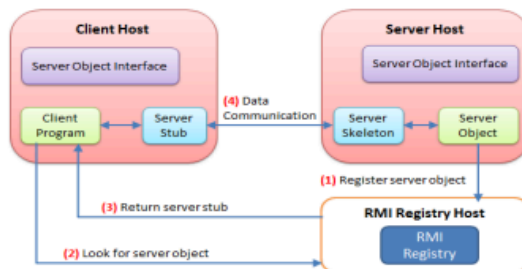


Figure 1: RMI Registry Process



CODE:

MyServer.java

```
import java.rmi.*;
import java.rmi.server.*;
import java.rmi.registry.*;

// Remote Object Implementation
class MyImpl extends UnicastRemoteObject implements MyInterface {

    public MyImpl() throws RemoteException {
        super();
    }

    public double add(double a, double b) throws RemoteException {
        return a + b;
    }
}
```

```

    }
}

// Server Class
public class MyServer {

    public static void main(String args[]) {

        try {

            // Create Remote Object
            MyImpl obj = new MyImpl();

            // Bind Object
            Naming.rebind("AddService", obj);

            System.out.println("Server is Ready...");
        }
        catch(Exception e) {
            System.out.println(e);
        }
    }
}

```

MyClient.java

```

import java.rmi.*;

public class MyClient {

    public static void main(String args[]) {

        try {

            String server = (args.length > 0) ? args[0] : "localhost";

            MyInterface obj = (MyInterface) Naming.lookup(
                "rmi://" + server + "/AddService");
            double op1=Double.parseDouble(args[1]);

```

```

        double op2=Double.parseDouble(args[2]);
        double op3 =Double.parseDouble(args[3]);

        System.out.println("Addition:" + obj.add(op1, op2));
        System.out.println("Addition:" + obj.add(op2, op3));
        System.out.println("Addition:" + obj.add(op3, op1));

    }
    catch(Exception e) {
        System.out.println(e);
    }
}
}

```

MyInterface.java

```

import java.rmi.*;

interface MyInterface extends Remote{
    double add(double a,double b) throws RemoteException;
}

```

OUTPUT:

```

E:\128003214 CN\exp3>javac *.java

E:\128003214 CN\exp3>start rmiregistry

E:\128003214 CN\exp3>javac MyServer
error: Class names, 'MyServer', are only
1 error

E:\128003214 CN\exp3>java MyServer
Server is Ready...
|

```

```
E:\128003214 CN\exp3>java MyClient localhost 700 800 300
Addition:1500.0
Addition:1100.0
Addition:1000.0
```

RESULTS:

When the client is executed with inputs 700 800 300, the server correctly returns the three addition results: 1500.0, 1100.0 and 1000.0.

The remote method add() is successfully invoked from the client through the RMI registry without any communication error.

INFERENCE:

RMI allows a client program to call methods on a remote object as if it were a local object. The registry acts as a naming service that binds and looks up the remote object, enabling distributed communication between client and server.